Template for the report:

PA2579 – Adaptive-lean software testing

Assignment 2 – Unit testing

Student name: Jacob Hellgren

Part 1 - Report:

Section 1: Project Description

The project used for this assignment was a smaller application made with Java. The application was developed as a part of this course and assignment. The reason for the project is to get practical experience of unit testing by implementing and testing a simple application component. The system that is being tested is a Java class called WeightHelper, its task being calculating Body Mass Index (BMI) based on a person's height and weight and returning the correct weight category. The categories measured are underweight, normal weight, overweight, and obesity. These categories are predefined BMI categories.

The project focuses on health-oriented calculations, specifically BMI. The application is simple but exemplifies how unit testing methods are used practically. The project was individually developed and tested. The focus was to understand how unit testing can be used to verify functionality and improve overall code quality. It was a new project as the application was coded for this assignment. The development method was a simple iterative method. The functionality was implemented, and then unit tests were made to verify that the application worked as intended. This made it possible to find possible errors in logic and improve the code.

Section 2: Roles and Responsibilities

In this project, as mentioned earlier, it was developed in an individual manner. This means that one developer held more than one role during the project, as a developer and tester. As a tester, I had the responsibility of designing and implementing unit tests that could verify the outputs. This included creating a test case based on techniques such as Boundary Value Analysis and Equivalence Partitioning. These techniques were used to identify important test values, especially those close to the BMI categories. Automated tools were used to generate even more test cases. In this project, EvoSuite was used to automatically create unit tests based on the system structure. These were used to compare manually designed test cases and automatic test cases.

As the application was created only as a part of this course, there was no external customer or product owner involved in the development process. The functionality requirements were therefore defined by those of the assignment, correctly implementing and exploring different methods of unit testing.

Section 3: Unit Test Activities and Practices

Unit testing was a central piece of the development process. After implementing the WeightHelper class, unit tests were created. They were written using the JUnit framework, which allowed them to be automatically run and verified that the expected results were returned.

The test cases were designed manually using the test techniques, Boundary Value Analysis (BVA) and Equivalence Partitioning (EP). They were used to identify relevant test values for the BMI classification. BMI is categorised into specific values 18.5, 25, and 30. By using BVA, test cases were created for values just under the lower and over the threshold to ensure that the classification worked correctly.

On top of the manual tests, automated test generation was also used. EvoSuite was used to generate even more tests based on the program structure. EvoSuite analyses the code and tries to automate the making of test cases that reach a high code coverage. These tests were used as a complement to the manual tests to examine how automated tests differ in comparison to manually designed tests.

The development process was iterative, where the functionality was implemented first and then verified with tests. Even if the project did not use a strict test-driven development process, tests were used continuously to make sure that the implementation worked correctly. The combination of manually designed tests and automatically generated tests contributed to increasing the reliability of the program and gave a better understanding of how different test strategies can be used practically.

Section 4: Environment and Tools

The development and testing of the system were made with different tools and environments that support unit testing and automated test generation. The programming language used was Java, and the development environment, IntelliJ IDEA, was used to write and compile the code as well as run the unit tests. This contributed to an effective development process.

For the manual tests, JUnit was used, which is an xUnit test tool for Java. JUnit made it possible to structure the test cases and automatically run the tests to verify that the methods in the WeightHelper class worked correctly. Through JUnit, the expected results could be compared to the actual results, which made it easier to identify errors in the programming logic.

Other than the manually created tests, EvoSuite was used to automate test generation. EvoSuite generates unit tests automatically by analysing the program structure and tries to reach high code coverage. The generated tests were used as a complement to the manual tests and to compare the differences between manually designed and automatically generated test cases.

The project used a command tool to run EvoSuite and handle dependencies, like EvoSuite's runtime library. Frameworks for mocking and static code analysis were not used in this project.

Section 5: Discussion and Concluding Remarks

This project gave an introduction to how unit tests are used practically and how it is used to improve the quality of the program. From a lean development perspective, the development is about reducing waste and creating a valuable product as fast as possible. Unit tests contribute to this by discovering errors early, which minimises the risk for larger problems later in the development process. In this project, the manual tests made it possible to quickly control that the BMI calculation and the classification worked as they should.

The manual tests that were created with Boundary Value and Equivalence Partitioning were effective because they focused on important limit values. These tests were easy to understand and had a clear connection to the function of the application. At the same time, the usage of EvoSuite to generate tests can contribute to a higher code coverage and help find cases that are not always thought of in manual test design.

There are, however, disadvantages to automated tests from a lean perspective. Many of the test cases that are generated by EvoSuite are hard to read and understand, which can make it harder to maintain. This can create some waste by making it more complex and time-consuming. Therefore, it might be more effective to combine manual and automated tests, where manual tests are used to cover important scenarios and automated tests are used to increase coverage.

If the project were to develop further, it would be interesting to work with a Test Driven Development (TDD) method. This means that the tests are written before the code is implemented, which can lead to better structure. The project application was not very large; had it been bigger, it would have been possible to implement continuous integration to automatically run tests for every change to the code. This highlights the trade-off between readability and coverage when comparing manual and automated test generation.